Dynamic Memory Configuration and Reconfiguration on FPGA

Hardware Implementation on Cyclone V FPGA

By: Sonakshi Agrawal (24BVD1006), Katakam Shrihita (24BVD1069), Sanjana Chejeti (24BVD1033)

1. INTRODUCTION

In modern SoC (System on Chip) designs, multiple hardware components such as CPU, GPU, SYS and DMA engines share a common DDR memory pool. Without a proper memory management system, one component can exhaust all available memory pages, causing other components to stall or fail. This project implements a fully hardware-based dynamic DDR memory allocator in SystemVerilog. The allocator divides DDR memory into independent regions — one per hardware component — each backed by a circular FIFO of free page addresses. A reconfiguration FSM continuously monitors region utilization and autonomously transfers pages between regions to prevent starvation, with zero software involvement.

2. SYSTEM OVERVIEW

- DDR Memory Model — simulated address space divided into N regions
- Region Manager — tracks which region owns which address blocks
- Per-Region FIFO — circular queue of free DDR addresses per region (CPU, GPU, DMA, etc.)
- Allocator — pops from a region's FIFO on allocation request
- Reclaimer — pushes freed addresses back to the tail
- Dynamic Reconfiguration — resizes FIFO capacity between regions at runtime
- Testbench — drives all scenarios including overflow, reconfig, and starvation

3. CODE LAYOUT

File	Role
region_fifo.sv	Circular FIFO holding free DDR page addresses per region
region_manager.sv	FSM managing 4 FIFOs with alloc, free, and reconfig logic
ddr_mem_allocator.sv	Top-level: Init FSM + MUX + region manager
tb_ddr_mem_allocator.sv	8-scenario self-checking testbench
top_wrapper.sv	FPGA wrapper connecting buttons and LEDs to allocator

4. DETAILED ARCHITECTURE

4.1 Circular FIFO (region_fifo.sv)

Each memory region is backed by a circular FIFO implemented as a fixed-size array of DDR page addresses. The FIFO supports three operations at once:

- PUSH (free): A freed page address is written at wr_ptr, which then advances. Count increments.
- POP (alloc): A page address is read from rd_ptr, which then advances. Count decrements.
- STEAL (reconfig): Identical to POP but triggered by the reconfiguration FSM, not a user request.

The circular nature ensures that wr_ptr and rd_ptr wrap around to index 0 after reaching MAX_DEPTH-1, so the array slots are reused indefinitely without overflow.

➔ FIFO Parameters

Parameter	Value	Description
ADDR_W	32	Width of each DDR page address (supports 4GB address space)
MAX_DEPTH	64	Maximum free pages the FIFO can hold per region
PTR_W	6	Pointer width = log2(64). rd_ptr and wr_ptr range 0 to 63
CNT_W	7	Count width = log2(64+1). Holds values 0 to 64 inclusive

4.2 Region Manager (region_manager.sv)

It handles two responsibilities simultaneously:

- Alloc/Free routing: When alloc_req[i] arrives, it drives pop_valid to FIFO i and returns the page address with alloc_ack. When free_req[i] arrives, it drives push_valid to FIFO i.
- Utilization monitoring: Every clock cycle, a combinational block scans all four region counts to identify a hungry region (count below LOW_THRESH=4) and a rich region (count above HIGH_THRESH=10). If both exist, the reconfig FSM is triggered.

4.3 Reconfiguration FSM

The reconfig FSM operates in three states to move pages from rich regions to hungry regions:

- ST_IDLE: Monitors counts every cycle. When a hungry and rich pair is detected, captures src and dst indices and transitions to ST_STEAL.
- ST_STEAL: Asserts steal_valid on the source FIFO. On acknowledgment, captures the page address into held_addr and moves to ST_DONATE.
- ST_DONATE: Pushes held_addr into the destination FIFO. If the imbalance persists, loops back to ST_STEAL. Otherwise returns to ST_IDLE.

This entire reconfig operation happens in hardware within 2-3 clock cycles, with no processor or software involvement.

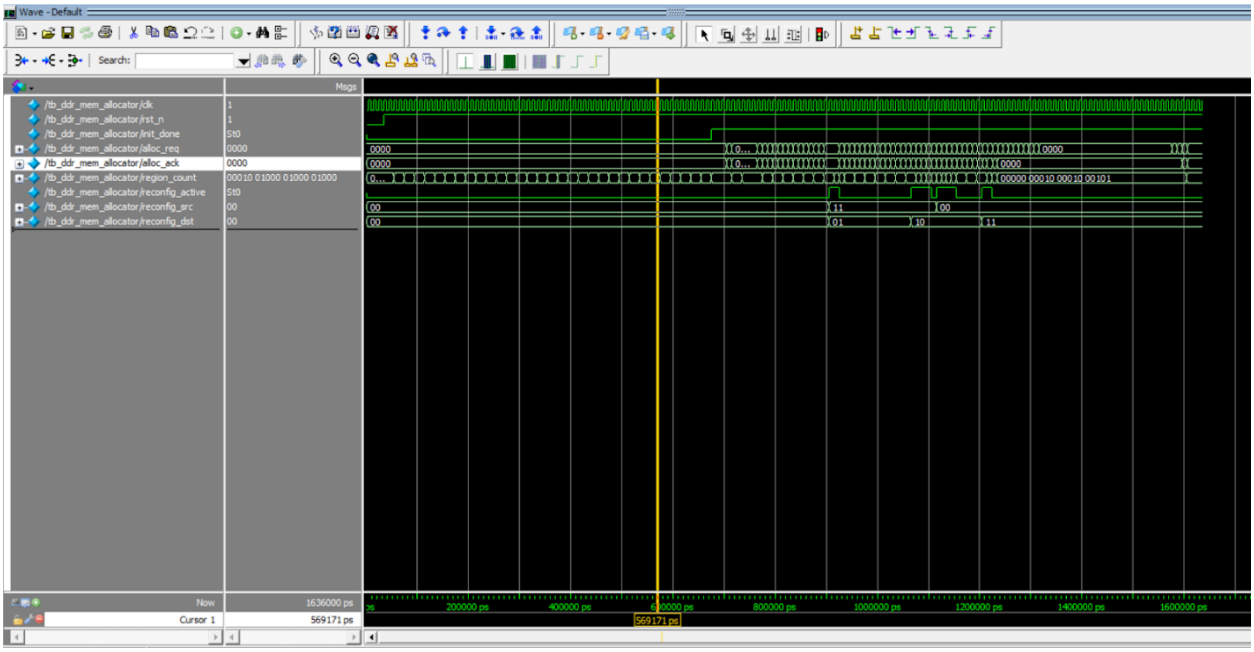
4.4 Init FSM and MUX (ddr_mem_allocator.sv)

On power-on or reset, the init FSM pre-loads each region FIFO with INIT_PAGES_PER_REGION=16 page addresses. The addresses are computed as:

BASE_ADDR + (region × 16 + page) × PAGE_SIZE

During initialization, a MUX blocks all external alloc and free requests by routing only the init FSM outputs to the region manager. The select signal is reconfig_en, which is tied to init_done. Once all four FIFOs are populated, reconfig_en asserts, the MUX switches to pass external requests, and the reconfig FSM is enabled.

5. MODELSIM WAVE OUTPUT:



6. FPGA HARDWARE DEMONSTRATION

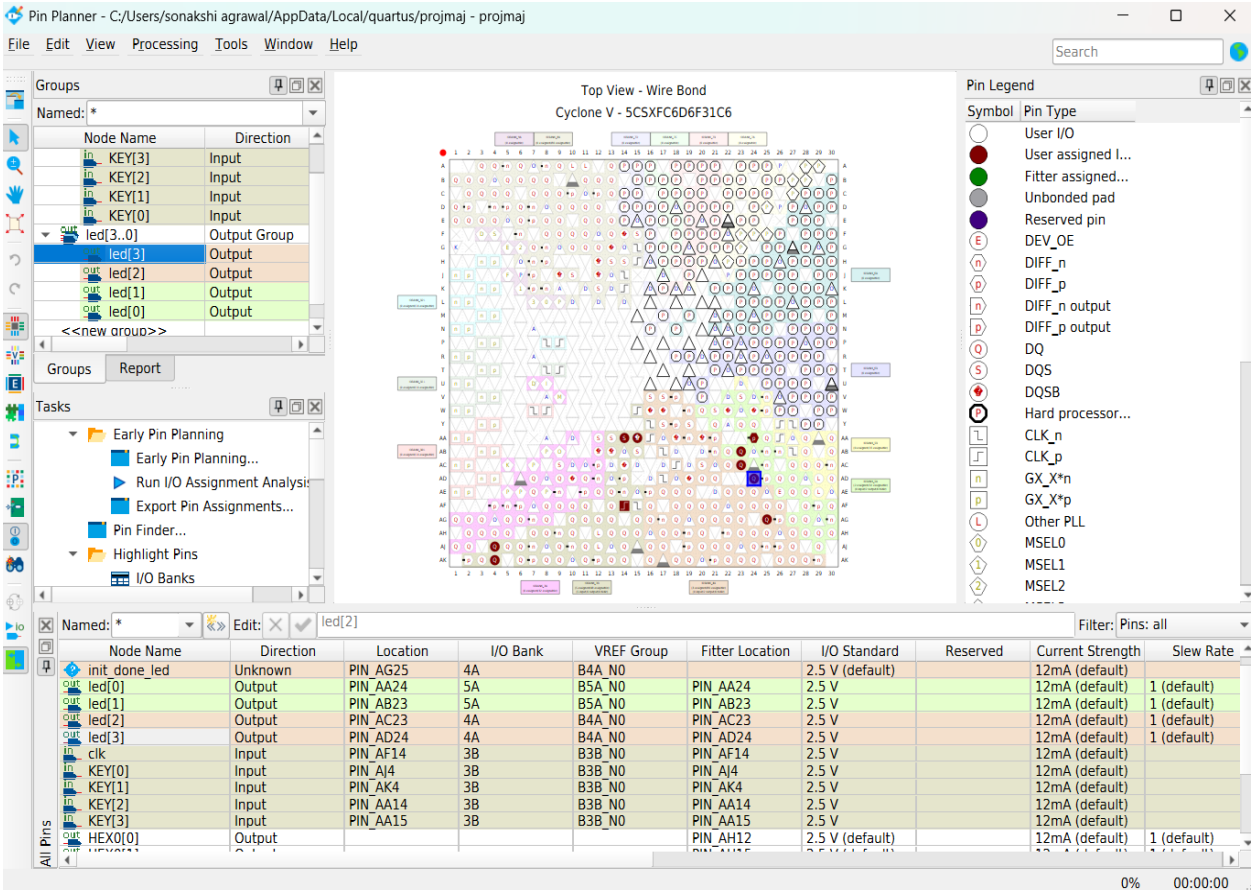
6.1 Target Device: Parameters

Parameter	Value
Board	DE10-Standard
FPGA	Intel Cyclone V — 5CSXFC6D6F31C6
Clock	50 MHz onboard oscillator
Reset	KEY[2] (active low)
I/O Standard	2.5V LVCMOS

6.2 LED Indicators

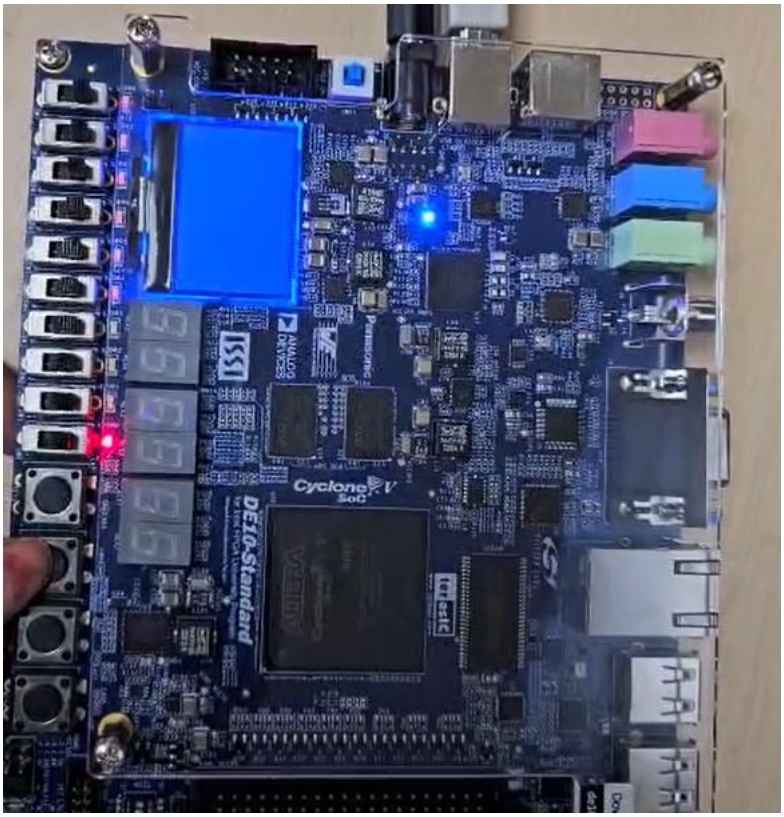
LED	Signal	Meaning
LED[0]	init_done	Memory pool initialized and ready. ON permanently after boot.
LED[1]	reconfig_active	Reconfig FSM actively moving a page between regions. Blinks briefly.
LED[2]	alloc_ack (latched)	At least one successful memory allocation has occurred. Latches ON.
LED[3]	alloc_fail (latched)	Allocation attempted on empty region. Lights if region exhausted (16+ presses).

6.3 Pin Planner

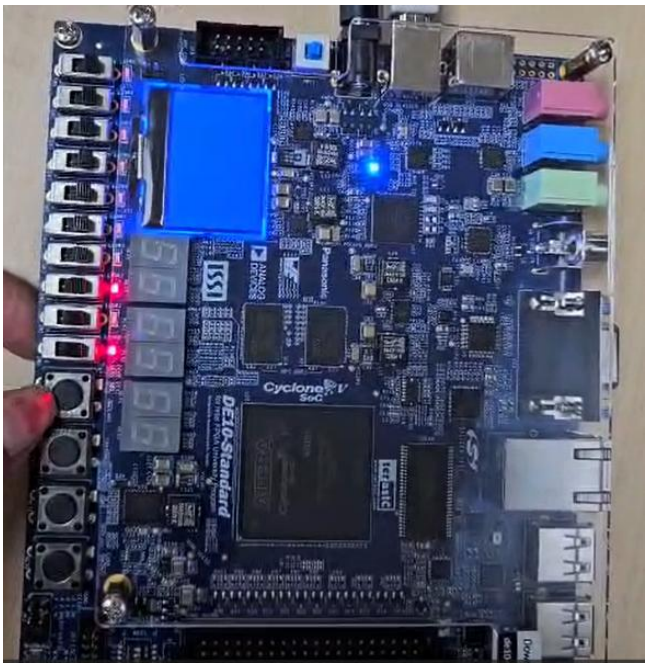


6.4 Observed Demo Flow

The following sequence was demonstrated on hardware:

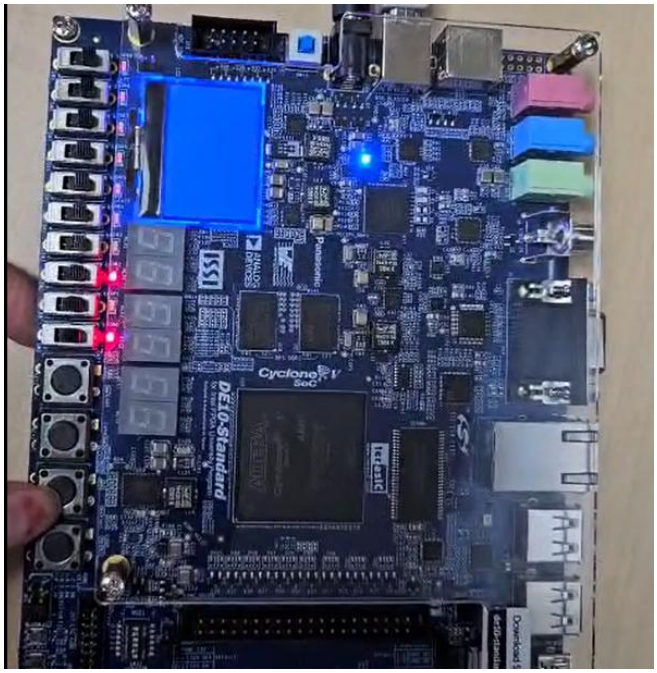


- KEY[2] pressed: System resets. LED[0] goes OFF, then comes back ON as init FSM repopulates all 4 FIFOs with 16 pages each (~130 clock cycles at 50MHz).

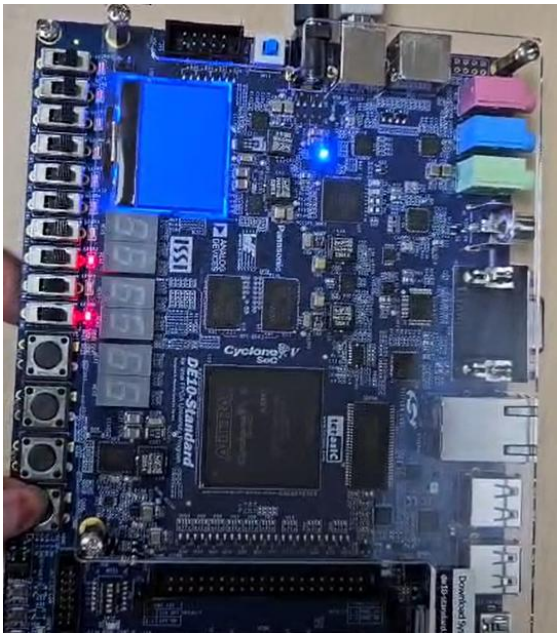


- KEY[3] pressed: alloc_req[0] fires, simulating CPU requesting a page. Region 0 FIFO pops one address. LED[2] latches ON. LED[0] and LED[2] glow together.

- KEY[2] pressed: Full reset and reinit. LED[2] clears. LED[0] blinks and returns.



- KEY[1] pressed: alloc_req[1] fires, simulating GPU requesting a page. Region 1 FIFO pops. LED[2] ON again with LED[0].



- KEY[0] pressed: alloc_req[2] fires, simulating DMA requesting a page. Region 2 FIFO pops. Same LED behavior.

LED[1] (reconfig_active) was not observed in the demo because only 1-2 allocs were performed per region, never dropping any region below the LOW_THRESH of 4. To trigger LED[1], a region would need to be drained below 4 pages while another remains above 10.

7. FUTURE DIRECTIONS

- For our next steps, this allocator will be connected to the ARM processor that is already built into the Cyclone V chip. This means instead of using buttons to simulate memory requests, the actual processor running real tasks like video playback, file transfers, or AI inference will send genuine alloc and free requests directly to the allocator.

- A monitoring block will be added on the FPGA side to watch the processor's memory usage in real time, so that when a background task suddenly needs more memory, the reconfig FSM moves pages to that region before it runs out, rather than waiting until it is already starving.

- The threshold values that decide when to rebalance, currently fixed in the code, will be made adjustable by the processor at runtime so the system can respond differently depending on how heavy the current workload is. The processor will also keep track of which addresses it borrowed and return them properly when done, completing the full borrow and return cycle that was not possible with just buttons.

- Finally the system will be scaled up from the small 16 page demo to handle megabytes (1000KB to 1024KB) of real DDR3 memory on the same board, making it a complete working memory manager that runs partly in hardware and partly in software.

8. CONCLUSION

This project successfully demonstrates a fully hardware-based dynamic DDR memory allocator implemented in SystemVerilog and synthesized on the Intel Cyclone V FPGA. The design provides single-cycle alloc and free operations across four independent memory regions, with an autonomous reconfiguration FSM that balances page distribution at runtime without any software intervention.

The FPGA demo validates initialization, allocation, and reset behavior through button inputs and LED indicators. The design is modular and parameterized, making it directly applicable to real SoC environments where CPU, GPU, and DMA engines compete for DDR memory pages at high speed.